

# API Testing Report

---

## Objective

The objective of this project is to perform **manual API testing** using **Postman** on the public JSONPlaceholder API. The goal is to verify core RESTful operations — GET, POST, PUT, PATCH, and DELETE — by validating response codes, body content, and response times.

## Tools & Technologies

- **Postman** – for sending and validating HTTP requests
- **JSONPlaceholder** – a free fake REST API for testing and prototyping
- **Manual Testing Approach** – for writing and executing test scenarios
- **Excel Sheet** – to maintain test cases and results

## API Endpoints Tested

| HTTP Method | Endpoint | Description                       |
|-------------|----------|-----------------------------------|
| GET         | /posts/1 | Fetch a specific post             |
| POST        | /posts   | Create a new post                 |
| PUT         | /posts/1 | Fully update an existing post     |
| PATCH       | /posts/1 | Partially update an existing post |
| DELETE      | /posts/1 | Delete a specific post            |

## Test Strategy

Each API was tested using Postman by:

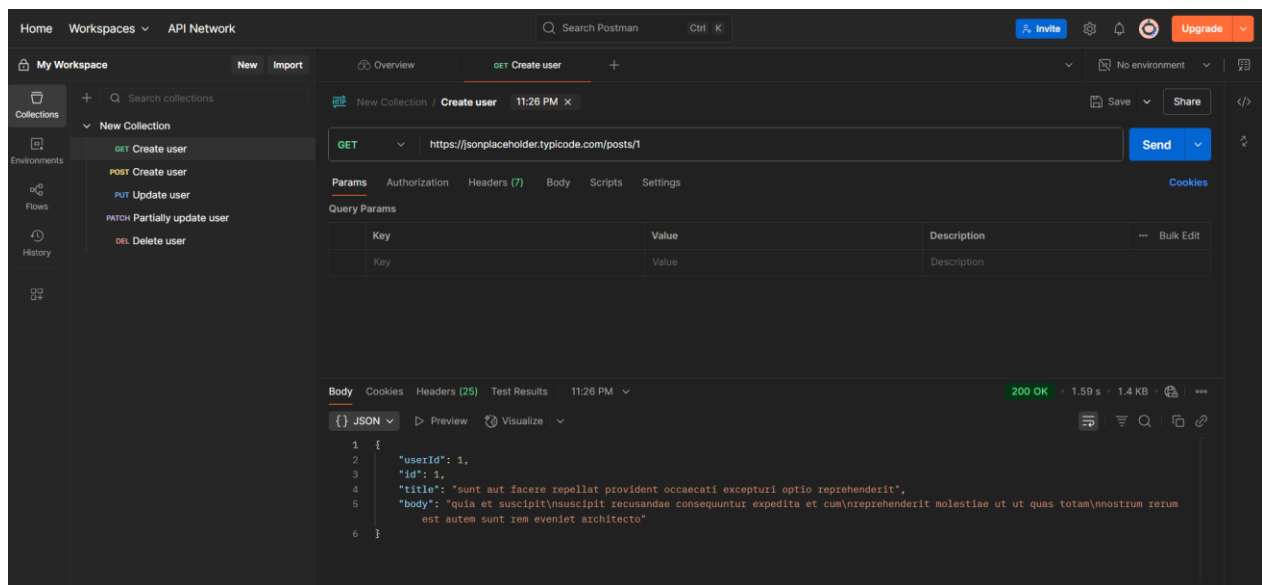
- Verifying the **HTTP status codes** (200, 201, 204)
- Validating the **response body** content and structure
- Checking **response times** (expected under 500ms)
- Performing **positive and negative testing** scenarios

# Summary of Test Cases

| Test Case ID | API Name            | Method | Expected Status | Pass/Fail |
|--------------|---------------------|--------|-----------------|-----------|
| TC001        | Get Post by ID      | GET    | 200 OK          | ✓ Pass    |
| TC002        | Create New Post     | POST   | 201 Created     | ✓ Pass    |
| TC003        | Update Post         | PUT    | 200 OK          | ✓ Pass    |
| TC004        | Delete Post         | DELETE | 200 OK / 204    | ✓ Pass    |
| TC005        | Partial Update Post | PATCH  | 200 OK          | ✓ Pass    |

Full detailed test cases are maintained in the attached Excel document.

## Screenshots



Home Workspaces API Network Search Postman Ctrl K Invite Upgrade

My Workspace New Import Overview GET Create user POST Create user

Collections + Search collections

Environments

Flows

History

New Collection

GET Create user

POST Create user

PUT Update user

PATCH Partially update user

DEL Delete user

POST ((baseURL)) /posts Send

Params Authorization Headers (9) Body Scripts Settings Cookies

Query Params

| Key | Value | Description | Bulk Edit |
|-----|-------|-------------|-----------|
| Key | Value | Description |           |

Body Cookies Headers (25) Test Results 201 Created · 357 ms · 1.26 KB Save Response

JSON Preview Visualize

```
1 {
2   "name": "mozpheus",
3   "job": "leader",
4   "id": 101
5 }
```

Postbot Runner Capture requests Cloud Agent Cookies Vault Trash

Click to go back, hold to see history Home Workspaces API Network Search Postman Ctrl K Invite Upgrade

My Workspace New Import Overview GET Create user POST Create user PUT Update user

Collections + Search collections

Environments

Flows

History

New Collection

GET Create user

POST Create user

PUT Update user

PATCH Partially update user

DEL Delete user

PUT ((baseURL)) /posts/1 Send

Params Authorization Headers (9) Body Scripts Settings Cookies

Query Params

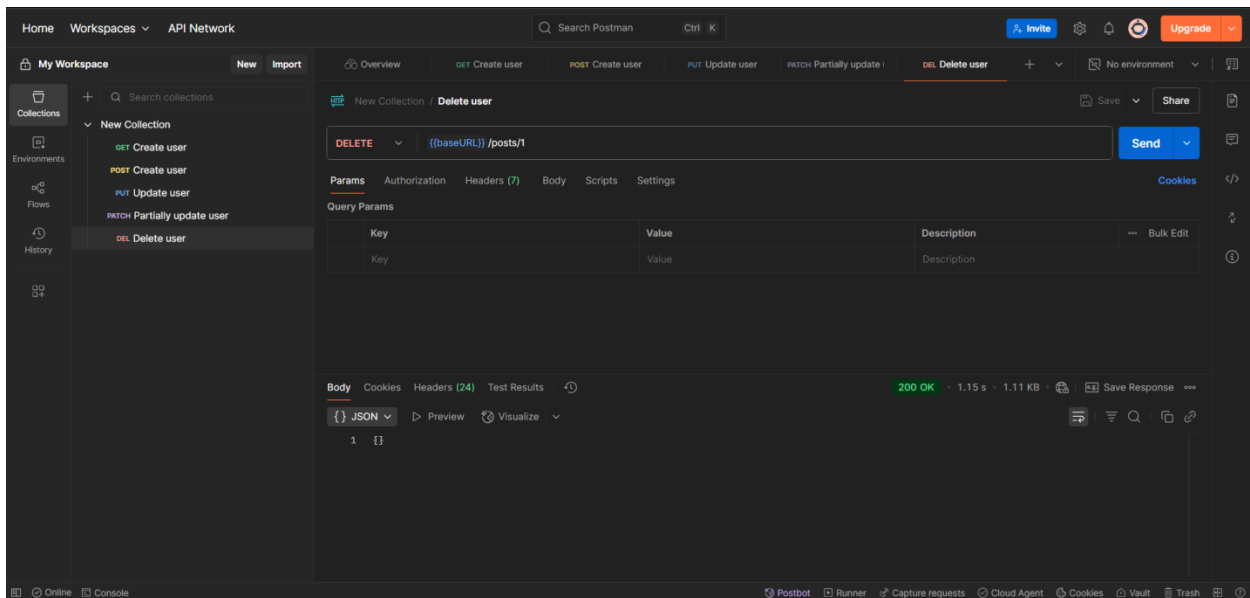
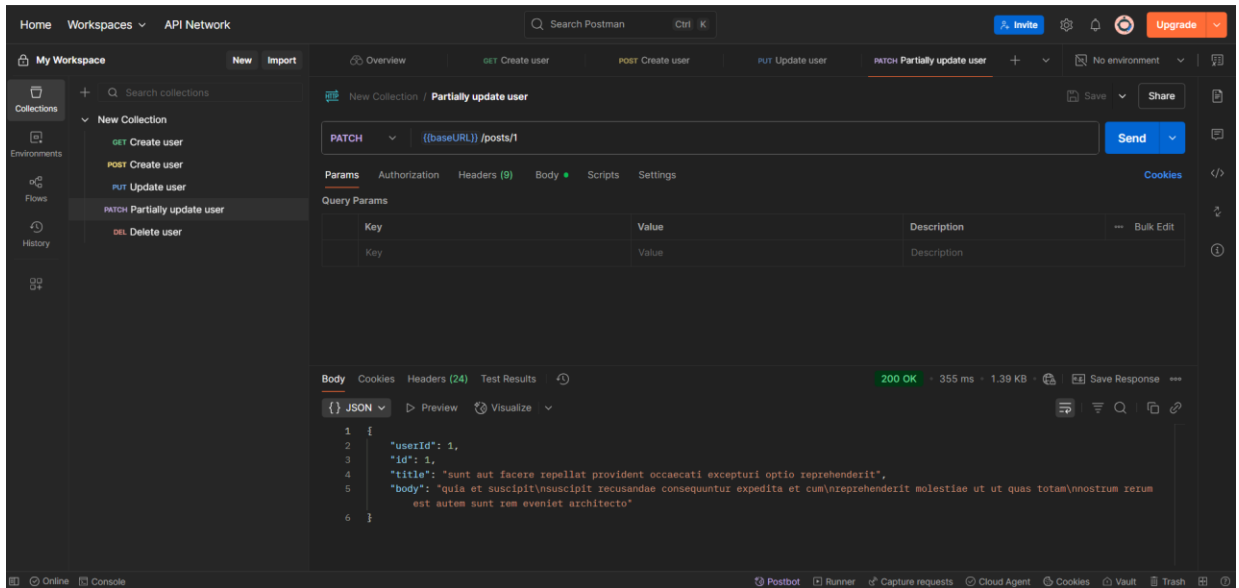
| Key | Value | Description | Bulk Edit |
|-----|-------|-------------|-----------|
| Key | Value | Description |           |

Body Cookies Headers (24) Test Results 200 OK · 408 ms · 1.39 KB Save Response

JSON Preview Visualize

```
1 {
2   "userId": 1,
3   "id": 1,
4   "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
5   "body": "quia et suscipit\nreuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
6 }
```

Postbot Runner Capture requests Cloud Agent Cookies Vault Trash



## Conclusion

Manual API testing using Postman provided a clear understanding of the behavior of RESTful APIs. All five HTTP methods were validated successfully using JSONPlaceholder. This testing confirms that:

- The API returns appropriate status codes
- The data formats are correct (JSON)
- The response structure is predictable and meets expectations